LAPORAN PENGUJIAN TEKNIS

Analisis Multi-Connection Speedtest CLI

Pengujian empiris perilaku jaringan Speedtest by Ookla CLI v1.2.0: apakah koneksi simultan diarahkan ke server berbeda, atau dikonsentrasikan pada satu server yang sama.

27	1	32
KONEKSI SIMULTAN	SERVER TUJUAN	BATAS ENGINE

Speedtest by Ookla 1.2.0.84 · Linux x86_64

4 NOVEMBER 2026 · SANDBOX LINUX · HONG KONG

1. Ringkasan Eksekutif

Pengujian dilakukan untuk menjawab satu pertanyaan spesifik: apakah Speedtest CLI (Ookla) versi 1.2.0 menggunakan multi connection saat melakukan tes kecepatan, dan bila ya — apakah koneksi-koneksi simultan itu diarahkan ke server yang berbeda-beda, atau justru dikonsentrasikan pada satu server yang sama. Pengujian dilakukan dengan cara menjalankan binary resmi yang diunduh dari installer yang disediakan, kemudian memantau seluruh socket TCP/UDP yang dibuka proses `speedtest` secara real-time memakai utilitas `ss -tunp` pada kernel Linux.

> **Temuan Utama**
>
> Speedtest CLI v1.2.0 **terbukti menggunakan multi connection**: hingga **27 koneksi TCP simultan** teramati selama fase download. Seluruh koneksi transfer data tersebut menuju **SATU server yang sama** (IP:port identik, hanya source port client yang berbeda-beda). Tidak ada transfer throughput yang dibagi ke beberapa server. Server yang berbeda hanya dilibatkan pada fase seleksi awal (ping ke 10 kandidat untuk memilih server dengan latency terbaik).

Jumlah koneksi juga terbukti adaptif terhadap kapasitas link: pada link cepat (hasil download 3,4–3,6 Gbps) engine membuka 20–27 koneksi paralel, sedangkan pada link yang lebih lambat pada sesi pengujian berbeda (hasil 626 Mbps) engine hanya membuka 6–8 koneksi. Konfigurasi internal yang terekstrak dari log verbose membenarkan mekanisme ini: `download.isScalingEnabled=true` dengan batas `maxConnections=32`.

2. Metodologi Pengujian

2.1 Lingkungan dan Komponen Uji

Binary Speedtest CLI diunduh langsung dari installer resmi Ookla yang diberikan, diekstrak tanpa modifikasi, dan dijalankan pada sandbox Linux x86_64 dengan kernel 5.10. Tabel berikut merangkum identitas komponen uji.

Komponen	Detail
Aplikasi uji	Speedtest by Ookla CLI 1.2.0.84 (build ea6b6773cf), Linux/x86_64-linux-musl
Sumber installer	install.speedtest.net/app/cli/ookla-speedtest-1.2.0-linux-x86_64.tgz (1,1 MB)
Sistem operasi	Linux x86_64, kernel 5.10.134, musl libc
ISP terdeteksi	Alibaba (datacenter Hong Kong)
Alat pemantau koneksi	ss -tunp (iproute2), polling setiap 150–200 ms selama proses berjalan
Analisis tambahan	Log verbose -vvv untuk ekstraksi konfigurasi engine; verifikasi DNS (nslookup/getent)

Tabel 1: Lingkungan pengujian

2.2 Prosedur

Tiga skenario pengujian dijalankan. **Skenario A** adalah tes standar dengan pemilihan server otomatis. **Skenario B** adalah tes standar kedua pada kondisi jaringan yang berbeda (untuk mengamati efek adaptive scaling). **Skenario C** adalah tes kontrol dengan server dipaksa memakai flag `-s 48446` (GSL Networks Hong Kong) guna membuktikan bahwa pola koneksi tidak berubah ketika pilihan server dimatikan. Pada setiap skenario, proses speedtest dijalankan di background dan socket milik proses tersebut dipotret berkala; hanya baris socket yang benar-benar dimiliki proses speedtest (terverifikasi lewat kolom PID) yang dihitung, sehingga koneksi proses lain di sandbox tidak mencemari data. Sebagai pembanding, flag `--single-connection` juga diuji keberadaannya.

3. Hasil Pengujian

3.1 Ringkasan Tiga Skenario

Ketiga skenario menghasilkan throughput dan jumlah koneksi yang berbeda, tetapi pola fundamentalnya identik. Hasil lengkapnya dirangkum pada tabel berikut.

Parameter	Skenario A	Skenario B	Skenario C (kontrol)
Mode server	otomatis	otomatis	dipaksa: -s 48446
Server terpilih	GSL Networks HK (id 48446)	HKBN HK (id 65463)	GSL Networks HK (id 48446)
IP:port server	202.165.70.3:8080	203.185.10.2:8080	202.165.70.3:8080
Download	3.476,15 Mbps	626,45 Mbps	3.605,37 Mbps
Upload	998,92 Mbps	633,62 Mbps	999,47 Mbps
Latency idle	4,86 ms	3,77 ms	5,19 ms
Puncak koneksi simultan (download)	**26 koneksi**	**8 koneksi**	**27 koneksi**
Koneksi fase upload	7 koneksi	6–7 koneksi	6–7 koneksi
Jumlah local port unik	35	15	36
Jumlah IP server tujuan (transfer data)	**1 (satu)**	**1 (satu)**	**1 (satu)**

Tabel 2: Ringkasan hasil tiga skenario pengujian

Baris terakhir adalah kunci jawaban pertanyaan pengujian: berapa pun banyaknya koneksi yang dibuka, transfer data throughput selalu mengarah ke tepat satu alamat IP server dengan port yang sama. Koneksi lain yang muncul hanyalah `140.248.166.219:443` (API konfigurasi/pelaporan hasil Ookla, bukan server tes) dan socket loopback internal — keduanya tidak membawa data throughput.

3.2 Uji Flag --single-connection

Flag `--single-connection` dicoba pada binary v1.2.0 dan hasilnya ditolak: aplikasi mencetak pesan penggunaan (usage) lalu keluar, artinya flag tersebut **tidak dikenali** pada versi ini. Dokumentasi resmi yang menyertai paket (speedtest.md/speedtest.5) juga tidak memuatnya. Kesimpulannya, pada v1.2.0 tidak ada opsi untuk memaksa koneksi tunggal — mode multi connection adalah satu-satunya perilaku yang tersedia. (Flag ini baru diperkenalkan pada rilis CLI versi 2.x.)

4. Analisis Perilaku Multi-Connection

4.1 Konfigurasi Engine yang Terekstrak

Menjalankan binary dengan verbosity maksimum (`-vvv`) membocorkan konfigurasi engine yang dikirim server konfigurasi Ookla. Cuplikan yang relevan:

```
{ "engine": { "isUploadFirst": false, "testDurationSeconds": 15, "threadCount": 4,
"packetSizeBytes": 32000000 }, "dynamic": { "download": { "isScalingEnabled": true,
"maxConnections": 32 } }, "loadedLatency": { "enableUdp": true, "enableTcp": true } }
```

Cuplikan konfigurasi engine dari log verbose (-vvv)

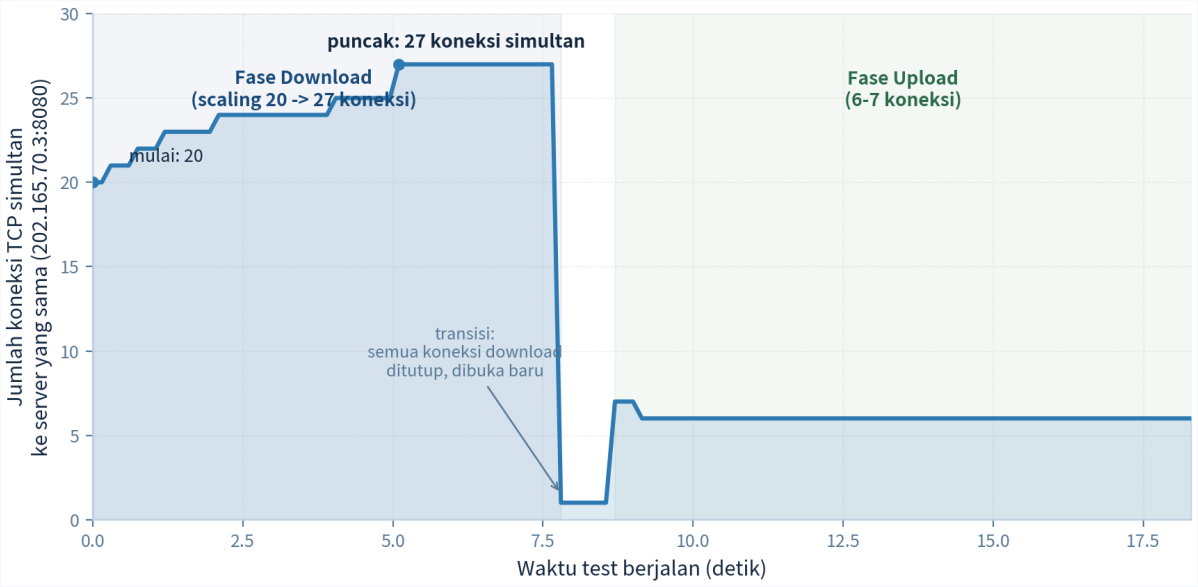
Tiga parameter menjelaskan semua yang teramati. Pertama, `maxConnections=32` adakah batas atas jumlah koneksi paralel untuk download — konsisten dengan 26–27 koneksi puncak yang teramati (link uji belum cukup cepat untuk menuntut 32). Kedua, `isScalingEnabled=true` berarti jumlah koneksi ditambah bertahap selama tes berjalan hingga throughput berhenti meningkat. Ketiga, `threadCount=4` menunjukkan empat thread worker yang mengelola seluruh koneksi tersebut. Log juga memperlihatkan mekanisme pemulihan: pesan `"Connection 1 failed. Added connection and continuing test."` membuktikan koneksi yang putus segera digantikan koneksi baru.

4.2 Kronologi Koneksi Selama Tes Berjalan

Pemantauan socket pada skenario C (kontrol) menghasilkan kronologi berikut. Jumlah koneksi dihitung dari socket berstatus established milik proses speedtest pada setiap interval polling 150 ms. Kronologi lengkap fase-fasennya dirangkum pada Tabel 3 sebelum divisualisasikan pada Gambar 1.

Fase	Waktu (approx.)	Jumlah koneksi	Aktivitas
Seleksi server	0 – ±2 detik	1 per kandidat (paralel, 10 server)	Ping latency ke 10 kandidat; 9 kalah langsung ditutup
Download	±2 – 8 detik	20 → 27 (scaling naik)	Transfer download 4,4 GB melalui koneksi paralel
Transisi	± detik ke-8	1	Seluruh koneksi download ditutup serentak
Upload	8 – 20 detik	6 – 7	Set koneksi baru dibuka ke server yang sama, transfer 1,3 GB
Penutupan	> 20 detik	3 → 0	Koneksi ditutup; hasil dikirim ke API Ookla (140.248.166.219:443)

Tabel 3: Kronologi fase test beserta jumlah koneksi teramati (skenario C)



Gambar 1: Jumlah koneksi TCP simultan terhadap waktu — skenario C (server dipaksa -s 48446)

Pola pada Gambar 1 membentuk tiga fase yang jelas. Fase download dimulai langsung dengan sekitar 20 koneksi, lalu bertambah satu-per-satu hingga puncak 27 koneksi pada detik ke-5 — inilah adaptive scaling yang dijanjikan konfigurasi engine. Menjelang detik ke-8 seluruh koneksi download ditutup serentak (garis turun tajam ke 1), lalu fase upload dibuka dengan set baru berisi 6–7 koneksi ke server yang sama hingga tes selesai. Perhatikan bahwa tidak ada satu pun momen ketika koneksi test mengarah ke lebih dari satu server.

4.3 Anatomi Lengkap: Seleksi Server vs Transfer Data

Diagram berikut merangkum keseluruhan anatomi koneksi Speedtest CLI dari awal hingga akhir, sekaligus menjawab pertanyaan inti pengujian ini.



Gambar 2: Anatomi koneksi — fase seleksi (kiri) dan fase transfer multi-connection (kanan)

Sisi kiri adalah satu-satunya momen ketika server yang berbeda dilibatkan: log verbose menunjukkan `"Opening socket to ..."` ke 10 alamat kandidat secara paralel — ini tes latency seleksi, bukan transfer throughput, dan hanya berlangsung satu-dua detik. Setelah satu pemenang dipilih (misalnya `server`

`selected: 28912`), sembilan kandidat lain langsung ditinggalkan. Sisi kanan adalah fase yang menentukan angka hasil: satu proses client membuka banyak socket TCP dengan source port berbeda-beda, tetapi seluruhnya menancap ke satu IP:port server yang sama. Melalui banyak koneksi inilah engine mengisi pipa bandwidth secara agresif — inilah sebabnya hasil speedtest hampir selalu lebih tinggi daripada unduhan single-stream biasa.

Perbandingan antar-skenario menambah satu insight: jumlah koneksi bukan angka tetap. Pada skenario B ketika jalur hanya mampu 626 Mbps, engine berhenti menambah koneksi pada angka 8 — menambah koneksi lagi tidak lagi menaikkan throughput, jadi scaling berhenti lebih awal. Pada skenario A dan C dengan jalur 3,4–3,6 Gbps, scaling terus berjalan hingga 26–27 koneksi. Dengan kata lain, jumlah koneksi adalah fungsi dari kapasitas link yang ditemukan selama tes berlangsung.

5. Kesimpulan

Pengujian memberikan jawaban tegas atas kedua pertanyaan. Pertama, **ya** — Speedtest CLI v1.2.0 menggunakan multi connection saat mengukur throughput: 6–8 koneksi paralel pada link ratusan Mbps, dan 20–27 koneksi paralel pada link multi-Gbps, dengan batas engine di 32 koneksi. Kedua, koneksi-koneksi simultan itu **seluruhnya menuju server yang sama** — satu IP, satu port — dan hanya dibedakan oleh source port di sisi client. Server yang berbeda hanya disentuh pada fase seleksi awal untuk memilih satu server dengan latency terbaik. Tidak ada mekanisme pembagian beban antar server pada versi yang diuji, dan tidak ada cara memaksa koneksi tunggal karena flag --single-connection belum ada di v1.2.0.

Implikasi praktisnya: hasil speedtest mencerminkan kemampuan agregat banyak koneksi TCP sekaligus (multiplexing), sehingga selalu lebih optimistis dibanding transfer satu koneksi seperti unduhan HTTP berjenjang tunggal atau satu stream SFTP. Bagi yang ingin mengukur performa koneksi tunggal — misalnya untuk menilai TCP tuning, window scaling, atau kualitas path satu sesi — angka speedtest bukan proksi yang tepat; gunakan iperf3 dengan mode -P 1 atau unduhan single-stream sebagai pembanding.

6. Lampiran: Perintah Replikasi Pengujian

Seluruh temuan pada laporan ini dapat direplikasi pada mesin Linux mana pun dengan urutan perintah berikut. Baris pertama mengunduh dan mengekstrak binary dari installer resmi; baris-baris berikutnya adalah varian pengujian yang dipakai pada tiap skenario, ditambah perintah pemantauan koneksi yang menjadi inti metodologi.

```
# 1. Unduh dan ekstrak installer resmi curl -sL -o speedtest.tgz \
https://install.speedtest.net/app/cli/ookla-speedtest-1.2.0-linux-x86_64.tgz tar -xzf
speedtest.tgz # 2. Skenario A/B - tes standar (server otomatis) ./speedtest
--accept-license # 3. Skenario C - kontrol dengan server dipaksa ./speedtest -s 48446
--accept-license # 4. Ekstrak konfigurasi engine (maxConnections, scaling) ./speedtest
-vvv --accept-license 2>&1 | grep -i scaling # 5. Pemantauan koneksi simultan saat tes
berjalan ./speedtest --accept-license & while kill -0 $!; do ss -tunp | grep speedtest;
sleep 0.15; done # 6. Daftar kandidat server terdekat beserta ID-nya ./speedtest -L
```

Urutan perintah replikasi pengujian

Sebagai pembanding, berikut cuplikan keluaran pemantauan yang sesungguhnya terekam pada momen puncak pengujian (detik ke-5 skenario C). Perhatikan kolom alamat peer: seluruhnya identik, dan hanya source port di sisi lokal yang berbeda dari satu baris ke baris lain.

```
$ ss -tunp | grep speedtest tcp ESTAB 0 0 21.0.7.203:42074 202.165.70.3:8080
users:((speedtest,pid=3604,fd=3)) tcp ESTAB 0 0 21.0.7.203:42030 202.165.70.3:8080
users:((speedtest,pid=3604,fd=6)) tcp ESTAB 0 0 21.0.7.203:42044 202.165.70.3:8080
users:((speedtest,pid=3604,fd=7)) tcp ESTAB 0 0 21.0.7.203:42148 202.165.70.3:8080
users:((speedtest,pid=3604,fd=18)) tcp ESTAB 0 0 21.0.7.203:42162 202.165.70.3:8080
users:((speedtest,pid=3604,fd=21)) tcp ESTAB 0 0 21.0.7.203:42172 202.165.70.3:8080
users:((speedtest,pid=3604,fd=23)) tcp ESTAB 0 0 21.0.7.203:42198 202.165.70.3:8080
users:((speedtest,pid=3604,fd=27)) tcp ESTAB 0 0 21.0.7.203:42206 202.165.70.3:8080
users:((speedtest,pid=3604,fd=28)) ... (total 27 baris serupa saat puncak, semua menuju
peer yang sama)
```

Cuplikan keluaran ss -tunp pada momen puncak (27 koneksi simultan, sumber: log pengujian)

Catatan interpretasi: pada keluaran pemantauan tersebut, setiap baris socket menuju server test menampilkan alamat peer yang identik pada seluruh baris (satu IP:port tujuan, source port berbeda) — itulah bukti langsung bahwa multi connection diarahkan ke satu server yang sama. Baris yang menunjuk ke port 443 atau alamat loopback bukanlah koneksi test, melainkan kanal API Ookla dan socket internal proses.

Keterbatasan pengujian perlu dicatat agar temuan dibaca dalam konteks yang tepat. Pertama, pengujian dilakukan dari sandbox datacenter (ISP Alibaba) dengan kandidat server selurnya di Hong Kong, sehingga latency sangat rendah dan perilaku di jaringan rumahan dengan latency lebih tinggi dapat sedikit berbeda — meski mekanisme dasarnya (multi connection ke satu server) tidak bergantung pada lokasi. Kedua, jumlah koneksi yang teramati adalah snapshot berkala setiap 150 ms; koneksi yang terbuka dan tertutup di antara dua snapshot mungkin tidak terhitung, sehingga angka sesungguhnya bisa sedikit lebih tinggi. Ketiga, kesimpulan berlaku spesifik untuk versi 1.2.0 yang diuji; perilaku engine dapat berubah pada rilis berikutnya, termasuk kemunculan flag --single-connection pada v2.x.